

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ
TRUYỀN THÔNG VIỆT HÀN
KHOA KỸ THUẬT MÁY TÍNH VÀ ĐIỆN TỬ**



ĐỒ ÁN CHUYÊN NGÀNH 2

**ĐỀ TÀI: THIẾT KẾ VÀ TRIỂN KHAI CƠ CHẾ MÃ HÓA
RSA/AES TRONG MÔ HÌNH CLIENT-SERVER**

Sinh viên thực hiện	: NGUYỄN ĐỨC HUY
Mã sinh viên	: 22NS024
Giảng viên hướng dẫn	: THS. LÊ KIM TRỌNG

Đà Nẵng, tháng 12 năm 2025

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ
TRUYỀN THÔNG VIỆT HÀN
KHOA KỸ THUẬT MÁY TÍNH VÀ ĐIỆN TỬ**



ĐỒ ÁN CHUYÊN NGÀNH 2

**ĐỀ TÀI: THIẾT KẾ VÀ TRIỂN KHAI CƠ CHẾ MÃ HÓA
RSA/AES TRONG MÔ HÌNH CLIENT-SERVER**

Sinh viên thực hiện	: NGUYỄN ĐỨC HUY
Mã sinh viên	: 22NS024
Giảng viên hướng dẫn	: THS. LÊ KIM TRỌNG

Đà Nẵng, tháng 12 năm 2025

LỜI CẢM ƠN

Trước hết, em xin chân thành gửi lời cảm ơn sâu sắc tới Ban Giám hiệu, Ban Chủ nhiệm Khoa và toàn thể quý thầy cô của VKU – nơi đã tạo điều kiện thuận lợi, môi trường học tập và nghiên cứu tốt nhất để em có thể thực hiện đề tài này.

Đặc biệt, em xin gửi lời cảm ơn trân trọng nhất đến thầy Lê Kim Trọng – giảng viên hướng dẫn của em. Trong suốt thời gian thực hiện đề tài và hoàn thiện luận văn, thầy đã dành nhiều thời gian, tâm huyết để hướng dẫn, chỉ bảo, trao đổi cùng em từ khâu định hướng đề tài, thu thập tư liệu, phân tích, tới khi hoàn thiện bản thảo cuối cùng. Những góp ý, nhận xét và động viên tận tình của thầy không những giúp em định hình rõ ràng hơn mục tiêu và nội dung nghiên cứu, mà còn giúp em học hỏi nhiều kỹ năng nghiên cứu, lập luận và trình bày học thuật — điều vô cùng quý giá đối với em.

Em cũng xin gửi lời cảm ơn đến các thầy cô trong khoa, những người đã dạy dỗ, truyền đạt kiến thức chuyên môn cũng như giúp đỡ, hỗ trợ em trong quá trình học tập và nghiên cứu.

Mặc dù em đã cố gắng hết sức, nhưng do hạn chế về kiến thức và kinh nghiệm thực tế, luận văn không thể tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý, chỉ dẫn từ thầy cô và các anh/chị để luận văn được hoàn thiện hơn. Em xin chân thành cảm ơn!

Đà Nẵng, ngày 25 tháng 12 năm 2025
Sinh viên thực hiện

NGUYỄN ĐỨC HUY

This image shows a full page of white paper with horizontal dotted lines, typical of primary-ruled notebook paper. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

THS. LÊ KIM TRỌNG

MỤC LỤC

MỞ ĐẦU.....	1
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT.....	3
1.1 Tổng quan về Mật mã học.....	3
1.1.1 Mã hóa đối xứng (Symmetric Key Encryption).....	3
1.1.2 Mã hóa bất đối xứng (Asymmetric Key Encryption).....	5
1.1.3 Bảng so sánh tóm tắt.....	6
1.2 Mã hóa đối xứng AES (Advanced Encryption Standard).....	6
1.2.1 Đặc tả kỹ thuật.....	7
1.2.2 Chế độ hoạt động CBC (Cipher Block Chaining).....	8
1.2.3 Kỹ thuật Đệm dữ liệu (Padding).....	8
1.3 Mã hóa bất đối xứng RSA (Rivest-Shamir-Adleman).....	9
1.3.1 Cơ chế tạo khóa (Key Generation).....	9
1.3.2 Quy trình Mã hóa và Giải mã.....	9
1.3.3 Chữ ký số RSA (Digital Signature).....	10
1.4 Chữ ký số (Digital Signature) và Hàm băm (SHA-256).....	10
1.4.1 Hàm băm mật mã (Cryptographic Hash Function) – SHA-256.....	10
1.4.2 Nguyên lý hoạt động của Chữ ký số.....	11
1.5 Mô hình Hệ mã lai (Hybrid Cryptosystem).....	12
1.5.1 Khái niệm.....	12
1.5.2 Cơ chế hoạt động: Kỹ thuật Mã hóa Phong bì (Digital Envelope).....	12
CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG.....	14
2.1 Phân tích yêu cầu chức năng.....	14
2.1.1 Yêu cầu chức năng phía Client.....	14
2.1.2 Yêu cầu chức năng phía Server.....	15
2.2 Phân tích yêu cầu phi chức năng.....	15
2.2.1 Yêu cầu về Bảo mật.....	15
2.2.2 Yêu cầu về Hiệu năng.....	16
2.3 Sơ đồ Kiến trúc Hệ thống.....	17
2.4 Kịch bản giao tiếp (Protocol Flow).....	18
2.4.1 Quy trình Bắt tay (Handshake) & Xác thực Server.....	18
2.4.2 Quy trình Đăng ký & Thiết lập Session Key.....	19
2.4.3 Quy trình Trao đổi tin nhắn (E2EE).....	20
2.5 Thiết kế cấu trúc gói tin.....	22
CHƯƠNG 3: TRIỂN KHAI ĐỀ TÀI.....	23
3.1 Quá trình khởi động.....	23
3.2 Quá trình xác thực Server và thiết lập phiên.....	23
3.3 Giao tiếp hệ thống.....	25
3.3.1 Kiểm tra danh sách Client online.....	25

3.3.2 Quá trình gửi tin nhắn.....	25
3.3.3 Quá trình nhận tin nhắn.....	26
3.3.4 Quá trình Server chuyển tiếp tin nhắn mã hóa.....	27
3.4 Chi tiết cấu trúc gói tin.....	27
3.4.1 Giai đoạn Bắt tay.....	27
3.4.2 Giai đoạn Đăng ký & Trao đổi khóa.....	28
3.4.3 Giai đoạn Giao tiếp Hệ thống.....	28
3.4.4 Giai đoạn Trao đổi tin nhắn.....	29
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	30
TÀI LIỆU THAM KHẢO.....	31

DANH MỤC HÌNH ẢNH

Hình 1.1: Tam giác CIA.....	3
Hình 1.2: Quá trình truyền tin trong mã hóa đối xứng.....	4
Hình 1.3: Quá trình truyền tin trong mã hóa bất đối xứng.....	5
Hình 1.4: Quy trình mã hóa AES.....	8
Hình 1.5: Quy trình mã hóa và giải mã RSA.....	10
Hình 1.6: Quy trình hoạt động của chữ ký số.....	12
Hình 1.7: Cơ chế hoạt động mã hóa Phong bì.....	13
Hình 2.1: Sơ đồ kiến trúc hệ thống.....	17
Hình 2.2: Quy trình Bắt tay (Handshake) & Xác thực Server.....	18
Hình 2.3: Quy trình Đăng ký & Thiết lập Session Key.....	19
Hình 2.4: Quy trình Trao đổi tin nhắn (E2EE).....	20
Hình 2.5: Luồng mã hóa và truyền tin E2EE.....	21
Hình 2.6: Cấu trúc gói tin.....	22
Hình 3.1: Quá trình khởi động Server.....	23
Hình 3.2: Quá trình khởi tạo Client và xác thực Server.....	23
Hình 3.3: Server từ chối kết nối nếu trùng username.....	23
Hình 3.4: Server tiếp nhận kết nối và phản hồi yêu cầu bắt tay.....	24
Hình 3.5: Client từ chối kết nối khi xác thực Server thất bại.....	24
Hình 3.6: Client hoàn tất xác thực và gửi yêu cầu đăng ký phiên làm việc.....	24
Hình 3.7: Server giải mã khóa và hoàn tất đăng ký người dùng.....	25
Hình 3.8: Truy vấn danh sách người dùng đang hoạt động.....	25
Hình 3.9: Gửi tin nhắn lần đầu và cơ chế tự động tìm kiếm khóa công khai.....	25
Hình 3.10: Quy trình nhận tin Giải mã và Xác thực chữ ký số.....	26
Hình 3.11: Hệ thống phát hiện và cảnh báo khi tin nhắn bị thay đổi.....	26
Hình 3.12: Server phân phối khóa và Chuyển tiếp tin nhắn mã hóa.....	27
Hình 3.13: Yêu cầu kết nối từ Client.....	27
Hình 3.14: Phản hồi từ Server (Chứa Chứng chỉ & Chữ ký).....	28
Hình 3.15: Client gửi Khóa phiên.....	28
Hình 3.16: Truy vấn danh sách Online.....	28
Hình 3.17: Truy vấn Public Key của người khác.....	29
Hình 3.18: Tin nhắn từ client này gửi tới client khác.....	29

DANH MỤC BẢNG

Bảng 1.1: So sánh mã hóa đối xứng và mã hóa bất đối xứng.....	6
Bảng 2.1: Yêu cầu chức năng phía Client.....	14
Bảng 2.2: Yêu cầu chức năng phía Server.....	15
Bảng 2.3: Yêu cầu bảo mật của hệ thống.....	16

MỞ ĐẦU

1. Lý do chọn đề tài

Trong quá trình học tập các môn liên quan đến an toàn thông tin và lập trình mạng, em nhận thấy rằng các kiến thức về mã hóa, đặc biệt là cơ chế kết hợp giữa mã hóa đối xứng và bất đối xứng, thường được trình bày ở mức lý thuyết và ít có cơ hội thực hành trực tiếp. Đề tài “Thiết kế và triển khai cơ chế mã hóa tin nhắn bằng RSA/AES trong mô hình client-server” giúp em có điều kiện củng cố kiến thức đã học, áp dụng vào bài toán thực tế và hiểu rõ hơn cách dữ liệu được bảo vệ khi trao đổi qua mạng. Đây là lý do em lựa chọn thực hiện đề tài này.

2. Mục tiêu nghiên cứu

- Hiểu và triển khai mã hóa AES và RSA trong truyền thông mạng: AES để mã hóa nội dung tin nhắn, RSA để bảo vệ khóa AES khi gửi tới người nhận.
- Thực hành chữ ký số và xác thực: Tạo chữ ký số cho mỗi tin nhắn, kiểm tra tính toàn vẹn và xác thực người gửi bằng public key.
- Xây dựng cơ chế handshake và xác thực server: Client kiểm tra server có hợp lệ thông qua certificate và chữ ký của CA.
- Thực hiện quản lý session key và public key: Mỗi client có session key riêng với server, public key được lưu trữ và phân phối một cách an toàn.
- Triển khai mô hình client-server

3. Phạm vi thực hiện

- Hệ thống chỉ hoạt động trên mạng nội bộ (localhost hoặc LAN).
- Giao tiếp dựa trên TCP socket và không triển khai giao thức HTTP hay HTTPS thực tế.
- Chỉ sử dụng CLI (Command-Line Interface), không phát triển GUI.
- Hệ thống chỉ triển khai mã hóa và xác thực cơ bản, chưa mở rộng các tính năng nâng cao như forward secrecy, chống replay attack hoặc truyền file.
- Server giữ vai trò điều phối, phân phối khóa và relay tin nhắn, không lưu trữ nội dung tin nhắn để đảm bảo tính riêng tư.

4. Phương pháp triển khai

- Nghiên cứu lý thuyết:

- + Mã hóa đối xứng (AES), mã hóa bất đối xứng (RSA), chữ ký số, certificate.
- + Cách hoạt động của hybrid encryption.
- Mô hình client–server và socket programming.
- Phân tích thiết kế hệ thống:
 - + Thiết kế giao thức trao đổi thông tin (packet format, actions).
 - + Thiết kế quá trình handshake và xác thực server.
 - + Thử nghiệm gửi nhận tin nhắn giữa các client.
- Kiểm thử và đánh giá:
 - + Kiểm thử bảo mật, tính năng, khả năng chạy đa client.
 - + Phân tích ưu nhược điểm và đưa ra hướng phát triển.

5. Kết quả đạt được

- Hiểu và triển khai thành công AES + RSA trong trao đổi dữ liệu.
- Hiểu cơ chế chữ ký số và xác thực nguồn gửi.
- Hiểu và thực hành handshake, xác thực server giống TLS.
- Xây dựng mô hình client–server chạy đa client song song.
- Hệ thống cho phép quản lý người dùng, kiểm soát trùng username, gửi tin nhắn bảo mật.
- Có thể giải thích rõ nguyên lý bảo mật, bao gồm Confidentiality, Integrity, Authentication, Non-repudiation.

6. Cấu trúc báo cáo

MỞ ĐẦU

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

CHƯƠNG 3: TRIỂN KHAI ĐỀ TÀI

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

TÀI LIỆU THAM KHẢO

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT

1.1 Tổng quan về Mật mã học

Mật mã học (Cryptography) là khoa học về việc bảo vệ thông tin bằng cách biến đổi nó sang một định dạng không thể đọc hiểu được (được gọi là bản mã - ciphertext) đối với những người không có quyền truy cập. Chỉ những người sở hữu khóa giải mã (secret key) mới có thể chuyển đổi bản mã trở lại thành thông tin ban đầu (bản rõ – plaintext).

Trong kỷ nguyên số, mật mã học đóng vai trò cốt lõi trong việc đảm bảo ba tính chất an toàn thông tin (CIA Triad):

- Tính bí mật (Confidentiality): Đảm bảo thông tin không bị lộ lọt cho bên thứ ba.
- Tính toàn vẹn (Integrity): Đảm bảo thông tin không bị sửa đổi trái phép.
- Tính xác thực (Authentication): Đảm bảo danh tính của các bên tham gia giao tiếp.



Hình 1.1: Tam giác CIA

Dựa trên cơ chế sử dụng khóa, các hệ mật mã hiện đại được chia thành hai loại chính: Mã hóa đối xứng và Mã hóa bất đối xứng.

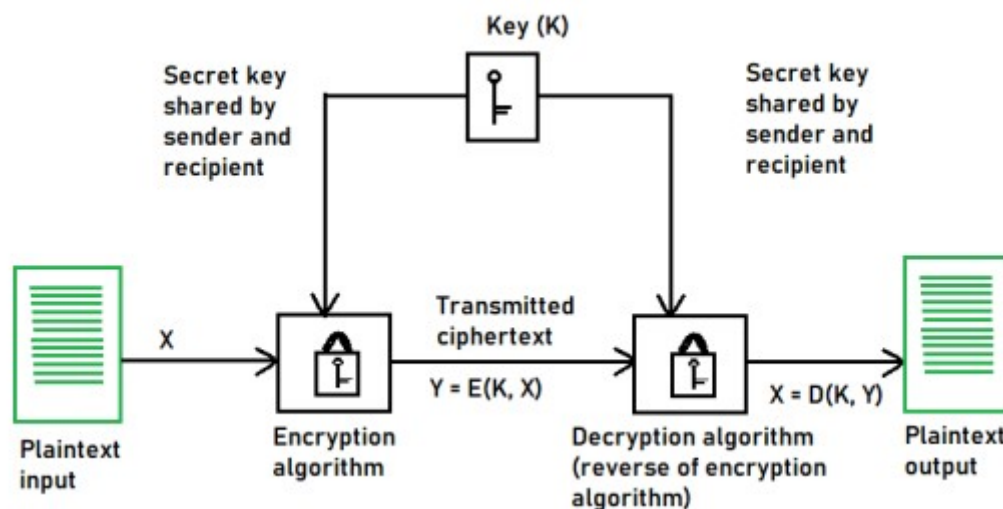
1.1.1 Mã hóa đối xứng (Symmetric Key Encryption)

Khái niệm: Mã hóa đối xứng (hay còn gọi là mã hóa khóa bí mật) là phương pháp mã hóa lâu đời và phổ biến nhất, trong đó người gửi và người nhận sử

dùng cùng một khóa duy nhất (secret key) để thực hiện cả hai quá trình mã hóa và giải mã.

Quá trình truyền tin diễn ra như sau:

- Bên gửi sử dụng khóa K để mã hóa bản rõ X thành bản mã Y : $Y = E(K, X)$
- Bên gửi truyền bản mã Y qua kênh truyền
- Bên nhận sử dụng cùng khóa K đó để giải mã bản mã Y về lại bản rõ X : $X = D(K, Y)$



Hình 1.2: Quá trình truyền tin trong mã hóa đối xứng

Ưu điểm:

- Tốc độ xử lý nhanh: Các thuật toán đối xứng thường có hiệu suất tính toán rất cao, tốn ít tài nguyên hệ thống, phù hợp để mã hóa khối lượng dữ liệu lớn (như nội dung tin nhắn, file, video).
- Độ phức tạp thấp: Dễ dàng triển khai trên phần cứng và phần mềm.
- Trong đề tài này, thuật toán AES (Advanced Encryption Standard) được lựa chọn làm đại diện cho mã hóa đối xứng để bảo vệ nội dung tin nhắn và phiên làm việc.

Nhược điểm:

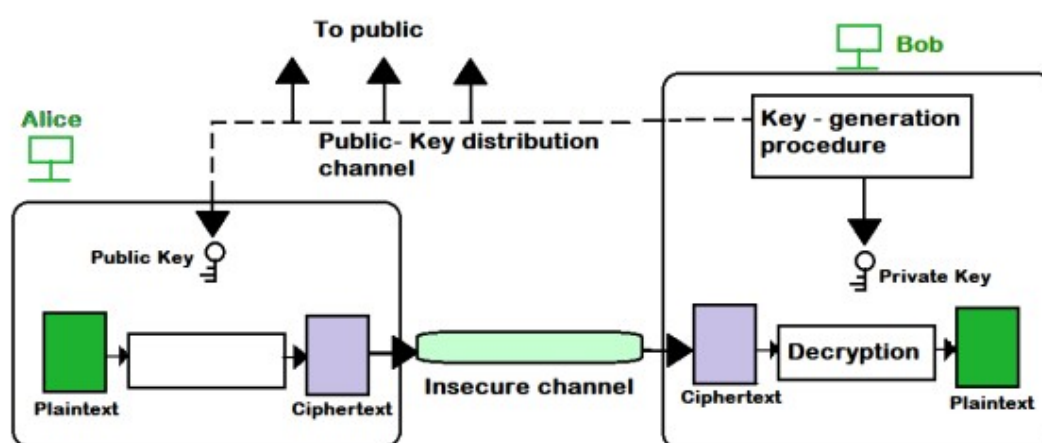
- Vấn đề phân phối khóa (Key Distribution Problem): Đây là điểm yếu chí mạng của mã hóa đối xứng. Làm thế nào để hai bên chưa từng gặp nhau có thể thống nhất một khóa bí mật K qua một môi trường mạng không an toàn mà không bị kẻ gian nghe lén? Nếu khóa bị lộ trong quá trình trao đổi, toàn bộ nội dung liên lạc sẽ bị xâm phạm.

- Quản lý khóa: Trong một mạng lưới có n người dùng, số lượng khóa cần quản lý sẽ tăng theo cấp số nhân ($n(n-1)/2$), gây khó khăn cho việc lưu trữ.

1.1.2 Mã hóa bất đối xứng (Asymmetric Key Encryption)

Khái niệm: Mã hóa bất đối xứng (hay còn gọi là mã hóa khóa công khai - Public Key Cryptography) ra đời để giải quyết bài toán phân phối khóa của mã hóa đối xứng. Phương pháp này sử dụng một cặp khóa có quan hệ toán học chặt chẽ với nhau:

- Khóa công khai (Public Key): Được công bố rộng rãi cho tất cả mọi người. Dùng để mã hóa dữ liệu hoặc kiểm tra chữ ký số.
- Khóa bí mật (Private Key): Được chủ sở hữu giữ bí mật tuyệt đối. Dùng để giải mã dữ liệu hoặc tạo chữ ký số.



Hình 1.3: Quá trình truyền tin trong mã hóa bất đối xứng

Có hai ứng dụng chính của mã hóa bất đối xứng:

- Bảo mật thông tin (Encryption): Người gửi dùng Public Key của người nhận để mã hóa. Chỉ có người nhận (người giữ Private Key tương ứng) mới có thể giải mã.
- Xác thực và Toàn vẹn (Digital Signature): Người gửi dùng Private Key của mình để ký lên thông điệp. Người nhận dùng Public Key của người gửi để xác minh rằng thông điệp đó thực sự do người gửi tạo ra và chưa bị chỉnh sửa.

Ưu điểm:

- Giải quyết vấn đề phân phối khóa: Không cần phải bí mật trao đổi khóa trước. Public key có thể được gửi công khai.

- Hỗ trợ xác thực (Non-repudiation): Cung cấp khả năng chống chối bỏ thông qua chữ ký số.
- Ứng dụng trong đề tài: Trong hệ thống này, thuật toán RSA đóng vai trò then chốt trong việc bảo vệ các khóa đối xứng (AES Keys). Cụ thể:
 - + Trao đổi khóa phiên (Session Key): Mã hóa khóa phiên khi Client đăng ký với Server.
 - + Trao đổi khóa tin nhắn (Message Key): Mã hóa khóa AES ngẫu nhiên đi kèm với từng gói tin nhắn, giúp bảo vệ nội dung chat theo cơ chế mã hóa đầu cuối (End-to-End Encryption).
 - + Xác thực: Tạo và kiểm tra chữ ký số (signature) để đảm bảo tính toàn vẹn.

Nhược điểm: Mã hóa bất đối xứng chậm hơn mã hóa đối xứng từ 100 đến 1000 lần do phải thực hiện các phép tính mũ trên các số nguyên rất lớn. Do đó, nó không thích hợp để mã hóa dữ liệu dài.

1.1.3 Bảng so sánh tóm tắt

Tiêu chí	Mã hóa đối xứng (Symmetric)	Mã hóa bất đối xứng (Asymmetric)
Số lượng khóa	1 khóa duy nhất (Secret Key).	2 khóa (Public Key & Private Key).
Tốc độ	Rất nhanh.	Rất chậm.
Mục đích sử dụng	Mã hóa dữ liệu lớn, kênh truyền thời gian thực.	Trao đổi khóa bí mật, Chữ ký số, Xác thực.
Đại diện tiêu biểu	DES, AES, 3DES, RC4.	RSA, ECC, Diffie-Hellman.

Bảng 1.1: So sánh mã hóa đối xứng và mã hóa bất đối xứng

1.2 Mã hóa đối xứng AES (Advanced Encryption Standard)

Tiêu chuẩn Mã hóa Nâng cao (AES) là một thuật toán mã hóa có độ tin cậy cao, được sử dụng để bảo mật dữ liệu bằng cách chuyển đổi dữ liệu sang định dạng không thể đọc được mà không cần khóa thích hợp. AES được Viện Tiêu chuẩn và Công nghệ Quốc gia (NIST) phát triển vào năm 2001. Ngày nay, AES được sử dụng rộng rãi vì mạnh hơn nhiều so với DES và DES ba lần mặc dù việc triển khai khó khăn hơn. Mã hóa AES sử dụng nhiều độ dài khóa khác nhau (128, 192 hoặc 256 bit) để cung cấp khả năng bảo vệ mạnh mẽ chống lại truy cập trái phép. Biện pháp bảo mật dữ liệu này hiệu quả và được triển khai

rộng rãi trong việc bảo mật thông tin liên lạc internet, bảo vệ dữ liệu nhạy cảm và mã hóa tệp. AES, nền tảng của mật mã hiện đại, được công nhận trên toàn cầu về khả năng bảo vệ thông tin an toàn khỏi các mối đe dọa mạng.

Về mặt cấu trúc, AES thuộc họ mã hóa khối (block cipher) hoạt động dựa trên mạng thay thế - hoán vị (Substitution-Permutation Network - SPN), cho phép thực hiện mã hóa và giải mã với tốc độ cao trên cả phần cứng lẫn phần mềm.

1.2.1 Đặc tả kỹ thuật

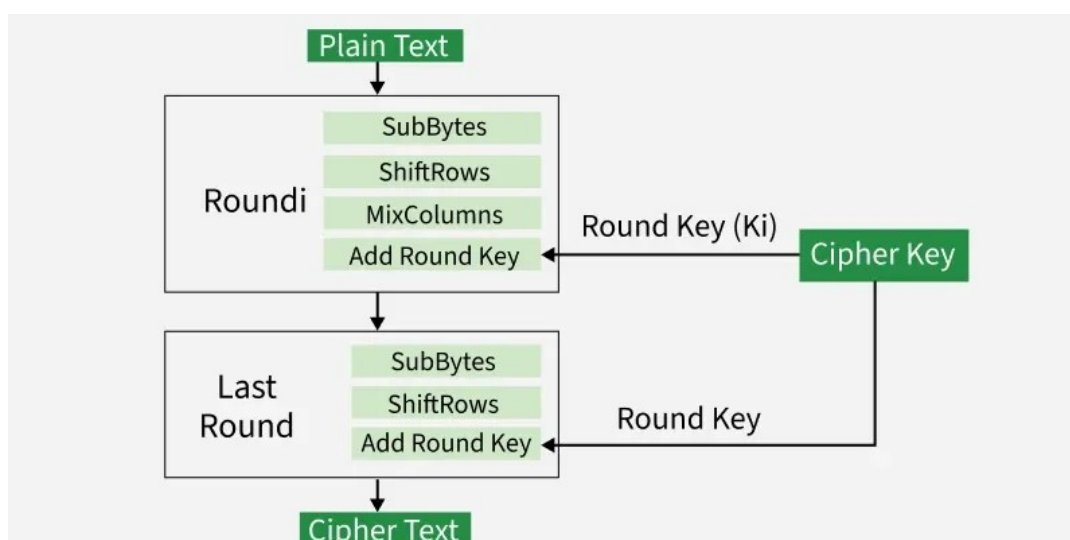
AES hoạt động trên các khối dữ liệu có kích thước cố định là 128 bit (16 bytes). Tùy thuộc vào độ dài của khóa bí mật, AES được chia thành ba biến thể với số lượng vòng lặp biến đổi (rounds) khác nhau:

- AES-128: Khóa 128 bit, thực hiện 10 vòng lặp.
- AES-192: Khóa 192 bit, thực hiện 12 vòng lặp.
- AES-256: Khóa 256 bit, thực hiện 14 vòng lặp.

Trong khuôn khổ đề tài này, hệ thống lựa chọn sử dụng AES-256 để đảm bảo mức độ bảo mật cao nhất có thể trước các cuộc tấn công vét cạn (brute-force).

Quá trình mã hóa một khối dữ liệu trải qua 4 phép biến đổi chính trong mỗi vòng lặp (ngoại trừ vòng cuối cùng):

- SubBytes: Thay thế phi tuyến tính mỗi byte dữ liệu bằng một byte khác thông qua bảng tra S-box.
- ShiftRows: Dịch chuyển vòng các hàng trong ma trận trạng thái.
- MixColumns: Trộn các cột để khuếch tán dữ liệu (bước này tạo ra sự thay đổi lớn trong bản mã từ một thay đổi nhỏ ở bản rõ).
- AddRoundKey: Cộng khóa (phép toán XOR) của vòng hiện tại vào ma trận trạng thái.



Hình 1.4: Quy trình mã hóa AES

1.2.2 Chế độ hoạt động CBC (Cipher Block Chaining)

Vì AES là mã hóa khối, việc mã hóa trực tiếp từng khối độc lập (chế độ ECB) sẽ làm lộ các mẫu dữ liệu lặp lại của bản rõ. Để khắc phục điều này và đảm bảo tính ngẫu nhiên của bản mã, đề tài áp dụng chế độ CBC (Cipher Block Chaining).

Nguyên lý của CBC là mỗi khối bản rõ sẽ được XOR với khối bản mã liền trước nó trước khi đi vào bộ mã hóa. Đối với khối đầu tiên, hệ thống sử dụng một Vector khởi tạo (Initialization Vector - IV) được sinh ngẫu nhiên.

Vai trò của IV trong hệ thống là cực kỳ quan trọng: nó đảm bảo rằng ngay cả khi hai người dùng gửi cùng một nội dung tin nhắn (ví dụ: "Hello"), hệ thống vẫn sẽ tạo ra hai chuỗi mã hóa hoàn toàn khác nhau. Điều này giúp ngăn chặn các cuộc tấn công phân tích mẫu (pattern analysis) và tấn công phát lại (replay attack).

1.2.3 Kỹ thuật Đệm dữ liệu (Padding)

Do AES yêu cầu dữ liệu đầu vào phải là bội số của kích thước khối (128 bit), nhưng độ dài tin nhắn thực tế của người dùng là ngẫu nhiên, nên kỹ thuật đệm (Padding) là bắt buộc.

Đề tài áp dụng chuẩn đệm PKCS#7. Cơ chế này hoạt động bằng cách thêm các byte giả vào cuối thông điệp cho đến khi đủ độ dài của một khối. Giá trị của các byte được thêm vào chính là số lượng byte còn thiếu. Khi giải mã, phía nhận sẽ dựa vào giá trị byte cuối cùng để loại bỏ phần đệm, khôi phục lại nội dung gốc chính xác.

1.3 Mã hóa bất đối xứng RSA (Rivest-Shamir-Adleman)

RSA là thuật toán mã hóa khóa công khai (asymmetric cryptography) đầu tiên và phổ biến nhất trên thế giới, được công bố vào năm 1977 bởi ba nhà khoa học Ron Rivest, Adi Shamir và Leonard Adleman tại Viện Công nghệ Massachusetts (MIT).

Độ an toàn của RSA dựa trên độ phức tạp tính toán của bài toán phân tích thừa số nguyên tố (Integer Factorization Problem). Cụ thể, việc nhân hai số nguyên tố lớn để tạo thành một hợp số là rất dễ dàng, nhưng việc phân tích một hợp số lớn trở lại thành hai thừa số nguyên tố cấu thành nó là bài toán cực khó, đòi hỏi năng lực tính toán khổng lồ mà các máy tính hiện tại chưa thể giải quyết trong thời gian thực.

1.3.1 Cơ chế tạo khóa (Key Generation)

Trong thuật toán RSA, mỗi thực thể tham gia hệ thống sẽ sở hữu một cặp khóa bao gồm:

- Khóa công khai (Public Key): Được công bố rộng rãi để bất kỳ ai cũng có thể sử dụng để mã hóa dữ liệu gửi đến chủ sở hữu.
- Khóa bí mật (Private Key): Được chủ sở hữu giữ bí mật tuyệt đối, dùng để giải mã dữ liệu hoặc tạo chữ ký số.

Đề tài sử dụng độ dài khóa RSA là 2048-bit. Đây là tiêu chuẩn an toàn được khuyến nghị bởi NIST hiện nay, đảm bảo sự cân bằng giữa độ bảo mật và hiệu năng tính toán.

1.3.2 Quy trình Mã hóa và Giải mã

Khác với mã hóa đối xứng, RSA sử dụng hai khóa khác nhau cho quá trình mã hóa và giải mã:

Mã hóa (Encryption):

Người gửi sử dụng Khóa công khai của người nhận để mã hóa thông điệp.

$$C = M^e \pmod{n}$$

Trong đó: M là bản rõ, C là bản mã, (e, n) là khóa công khai.

Giải mã (Decryption):

Người nhận sử dụng Khóa bí mật của mình để khôi phục thông điệp gốc.

$$M = C^d \pmod{n}$$

Trong đó (d, n) là khóa bí mật.



Hình 1.5: Quy trình mã hóa và giải mã RSA

Cơ chế này đảm bảo tính bí mật (Confidentiality): Ngay cả khi bản mã bị đánh cắp trên đường truyền, kẻ tấn công không thể giải mã được nếu không có khóa bí mật tương ứng.

1.3.3 Chữ ký số RSA (Digital Signature)

Ngoài chức năng mã hóa, RSA còn đóng vai trò cốt lõi trong cơ chế xác thực và đảm bảo toàn vẹn dữ liệu thông qua chữ ký số. Quy trình này hoạt động ngược lại với quy trình mã hóa:

- Ký (Sign): Người gửi sử dụng hàm băm (như SHA-256) để tạo ra giá trị băm (digest) của thông điệp, sau đó mã hóa giá trị băm này bằng Khóa bí mật của mình. Kết quả thu được là chữ ký số.
- Xác thực (Verify): Người nhận giải mã chữ ký bằng Khóa công khai của người gửi để lấy lại giá trị băm gốc, sau đó so sánh với giá trị băm tự tính toán từ thông điệp nhận được. Nếu hai giá trị trùng khớp, chứng tỏ thông điệp toàn vẹn và thực sự xuất phát từ người gửi.

1.4 Chữ ký số (Digital Signature) và Hàm băm (SHA-256)

Để đảm bảo an toàn thông tin trong môi trường mạng, việc mã hóa dữ liệu (giữ bí mật) là chưa đủ. Hệ thống cần phải có cơ chế để xác minh danh tính của người gửi và đảm bảo rằng dữ liệu không bị thay đổi trong quá trình truyền tải. Chữ ký số kết hợp với hàm băm mật mã chính là giải pháp cho vấn đề này.

1.4.1 Hàm băm mật mã (Cryptographic Hash Function) – SHA-256

Khái niệm: Hàm băm là một thuật toán toán học có chức năng chuyển đổi một chuỗi dữ liệu đầu vào (message) có độ dài bất kỳ thành một chuỗi ký tự có độ dài cố định, được gọi là "giá trị băm" (hash value) hoặc "bản tóm tắt thông điệp" (message digest).

Chuẩn SHA-256: Trong đề tài này, thuật toán SHA-256 (Secure Hash Algorithm 256-bit) thuộc họ SHA-2 được lựa chọn sử dụng. SHA-256 tạo ra một giá trị băm dài 256 bit (32 byte), thường được biểu diễn dưới dạng chuỗi 64 ký tự thập lục phân (hexadecimal).

Các tính chất quan trọng:

- Tính một chiều (Pre-image resistance): Từ giá trị băm, không thể suy ngược lại nội dung gốc. Điều này giúp bảo vệ nội dung thông điệp khi chỉ cần xác thực.
- Hiệu ứng lan truyền (Avalanche effect): Chỉ một thay đổi nhỏ nhất trong dữ liệu đầu vào (dù chỉ 1 bit) cũng sẽ dẫn đến sự thay đổi hoàn toàn khác biệt của giá trị băm đầu ra. Tính chất này là cốt lõi để phát hiện mọi hành vi sửa đổi dữ liệu trái phép (tampering).
- Khả năng chống va chạm (Collision resistance): Xác suất để tìm thấy hai thông điệp khác nhau có cùng một giá trị băm là cực kỳ thấp (gần như bằng 0), đảm bảo tính duy nhất của dấu vân tay điện tử.

1.4.2 Nguyên lý hoạt động của Chữ ký số

Chữ ký số là một cơ chế xác thực điện tử, đóng vai trò tương tự như chữ ký tay hoặc con dấu trong văn bản giấy, nhưng có độ bảo mật cao hơn nhiều. Nó dựa trên nền tảng của mã hóa bất đối xứng (RSA) kết hợp với hàm băm.

Quy trình hoạt động bao gồm hai giai đoạn:

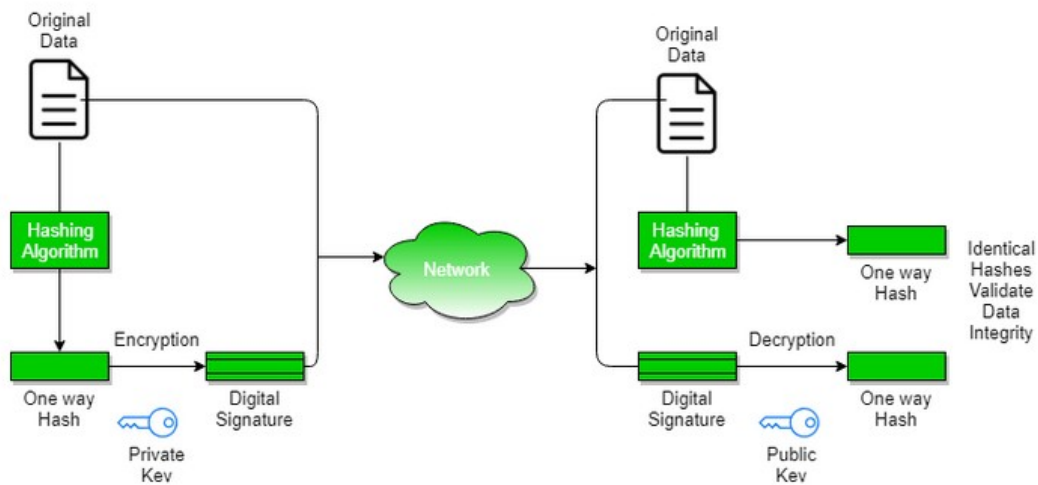
1. Giai đoạn Ký (Signing):

- Hệ thống sử dụng SHA-256 để băm thông điệp gốc, tạo ra bản tóm tắt (digest).
- Người gửi sử dụng Khóa bí mật (Private Key) của mình để mã hóa bản tóm tắt này. Kết quả thu được chính là "Chữ ký số".
- Chữ ký này được đính kèm vào gói tin và gửi đi cùng với dữ liệu.

2. Giai đoạn Kiểm thử (Verifying):

- Người nhận tách riêng phần dữ liệu và phần chữ ký số.
- Sử dụng Khóa công khai (Public Key) của người gửi để giải mã chữ ký số, thu được giá trị băm gốc (H1).
- Đồng thời, người nhận tự tính toán lại giá trị băm của phần dữ liệu nhận được bằng SHA-256 (H2). So sánh H1 và H2.

- Nếu trùng khớp, chứng tỏ dữ liệu toàn vẹn và thực sự xuất phát từ người nắm giữ khóa bí mật.



Hình 1.6: Quy trình hoạt động của chữ ký số

1.5 Mô hình Hệ mã lai (Hybrid Cryptosystem)

1.5.1 Khái niệm

Trong thực tế triển khai các hệ thống bảo mật, việc chỉ sử dụng đơn lẻ mã hóa đối xứng (AES) hoặc mã hóa bất đối xứng (RSA) đều bộc lộ những hạn chế nhất định:

- AES có tốc độ xử lý cực nhanh nhưng gặp khó khăn lớn trong việc phân phối khóa an toàn qua môi trường mạng.
- RSA giải quyết tốt vấn đề phân phối khóa nhưng lại có hiệu năng thấp và tốc độ xử lý chậm khi làm việc với khối dữ liệu lớn.

Hệ mã lai (Hybrid Cryptosystem) là giải pháp kiến trúc kết hợp ưu điểm của cả hai phương pháp trên: sử dụng tốc độ của mã hóa đối xứng cho việc bảo vệ dữ liệu và sử dụng tính linh hoạt của mã hóa bất đối xứng cho việc bảo vệ khóa. Đây là mô hình chuẩn mực đang được sử dụng trong các giao thức an toàn mạng phổ biến như SSL/TLS, SSH và PGP.

1.5.2 Cơ chế hoạt động: Kỹ thuật Mã hóa Phong bì (Digital Envelope)

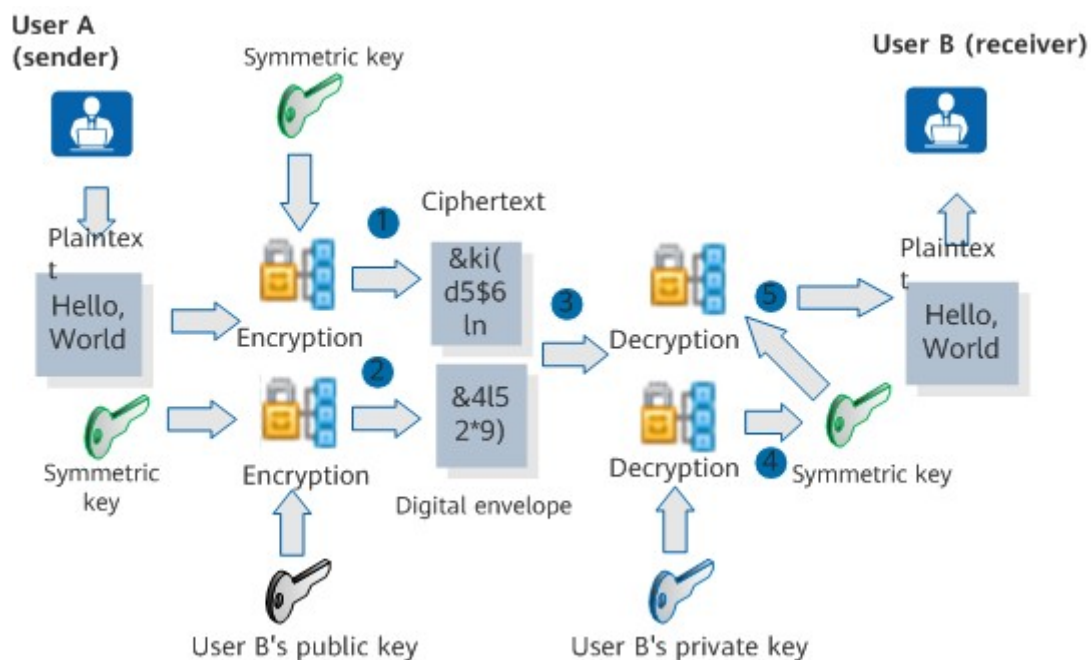
Nguyên lý cốt lõi của hệ mã lai được áp dụng trong đồ án dựa trên kỹ thuật "Mã hóa Phong bì". Quy trình hoạt động tổng quát như sau:

Bước 1 - Tạo khóa phiên: Bên gửi sinh ra một khóa đối xứng ngẫu nhiên (gọi là Session Key hoặc Message Key) chỉ dùng cho phiên làm việc đó hoặc tin nhắn đó.

Bước 2 - Mã hóa dữ liệu: Sử dụng khóa đối xứng vừa tạo để mã hóa nội dung thông điệp (dùng thuật toán AES).

Bước 3 - Mã hóa khóa: Sử dụng khóa công khai (Public Key) của người nhận để mã hóa chính cái khóa đối xứng đó (dùng thuật toán RSA).

Bước 4 - Đóng gói: Gửi cả "Thông điệp đã mã hóa" và "Khóa đối xứng đã mã hóa" cho người nhận.



Hình 1.7: Cơ chế hoạt động mã hóa Phong bì

CHƯƠNG 2: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1 Phân tích yêu cầu chức năng

2.1.1 Yêu cầu chức năng phía Client

STT	Chức năng	Mô tả
1	Thiết lập kết nối an toàn (Secure Handshake)	Gửi yêu cầu handshake đến Server, nhận chứng chỉ số và xác thực Server bằng Public Key của CA nhằm đảm bảo kết nối không bị giả mạo.
2	Đăng ký định danh người dùng	Cho phép người dùng nhập Username; Client tự động sinh cặp khóa RSA và khóa phiên AES, sau đó gửi thông tin đăng ký đã được mã hóa đến Server.
3	Tra cứu người dùng trực tuyến	Gửi yêu cầu truy vấn danh sách các người dùng đang online và hiển thị kết quả trên giao diện CLI.
4	Gửi tin nhắn riêng tư	Cho phép gửi tin nhắn theo cú pháp @username <nội dung>; thực hiện mã hóa nội dung bằng AES, mã hóa khóa AES bằng RSA và ký số bằng Private Key của người gửi.
5	Nhận và giải mã tin nhắn	Nhận gói tin từ Server, giải mã khóa AES bằng Private Key, giải mã nội dung tin nhắn và hiển thị ra màn hình.
6	Xác thực chữ ký số	Kiểm tra chữ ký số của người gửi để đảm bảo tính toàn vẹn và xác thực nguồn gốc tin nhắn.
7	Lắng nghe và xử lý sự kiện hệ thống	Duy trì kết nối, lắng nghe các thông báo hệ thống (user online/offline) và xử lý ngắt kết nối an toàn.

Bảng 2.1: Yêu cầu chức năng phía Client

2.1.2 Yêu cầu chức năng phía Server

STT	Chức năng	Mô tả
1	Quản lý kết nối đa luồng	Chấp nhận và xử lý đồng thời nhiều kết nối Socket từ các Client bằng cơ chế đa luồng (threading).
2	Phản hồi Handshake	Gửi chứng chỉ số của Server cho Client để phục vụ quá trình xác thực danh tính.
3	Xác thực và đăng ký người dùng	Giải mã Session Key, kiểm tra trùng lặp Username và lưu thông tin phiên làm việc của Client vào bộ nhớ tạm.
4	Quản lý phiên làm việc	Lưu trữ và quản lý Socket, Session Key, Public Key của từng Client trong suốt thời gian kết nối.
5	Phân phối khóa công khai	Cung cấp Public Key của người dùng theo yêu cầu để hỗ trợ mã hóa tin nhắn đầu cuối.
6	Chuyển tiếp tin nhắn	Định tuyến và chuyển tiếp nguyên vẹn các gói tin đã mã hóa từ người gửi đến người nhận mà không can thiệp nội dung.
7	Quản lý trạng thái người dùng	Theo dõi trạng thái online/offline của người dùng và cập nhật danh sách khi có thay đổi.
8	Thông báo hệ thống	Gửi danh sách người dùng online và thông báo khi có Client ngắt kết nối.

Bảng 2.2: Yêu cầu chức năng phía Server

2.2 Phân tích yêu cầu phi chức năng

2.2.1 Yêu cầu về Bảo mật

STT	Thuộc tính bảo mật	Mô tả yêu cầu	Cơ chế / Thuật toán áp dụng
1	Tính bí mật (Confidentiality)	Đảm bảo nội dung tin nhắn chỉ được giải mã tại thiết bị người nhận; Server và các bên trung gian không thể đọc được dữ liệu.	Mã hóa đầu cuối (E2EE); AES-256 (CBC) để mã hóa nội dung; RSA để mã hóa khóa AES.

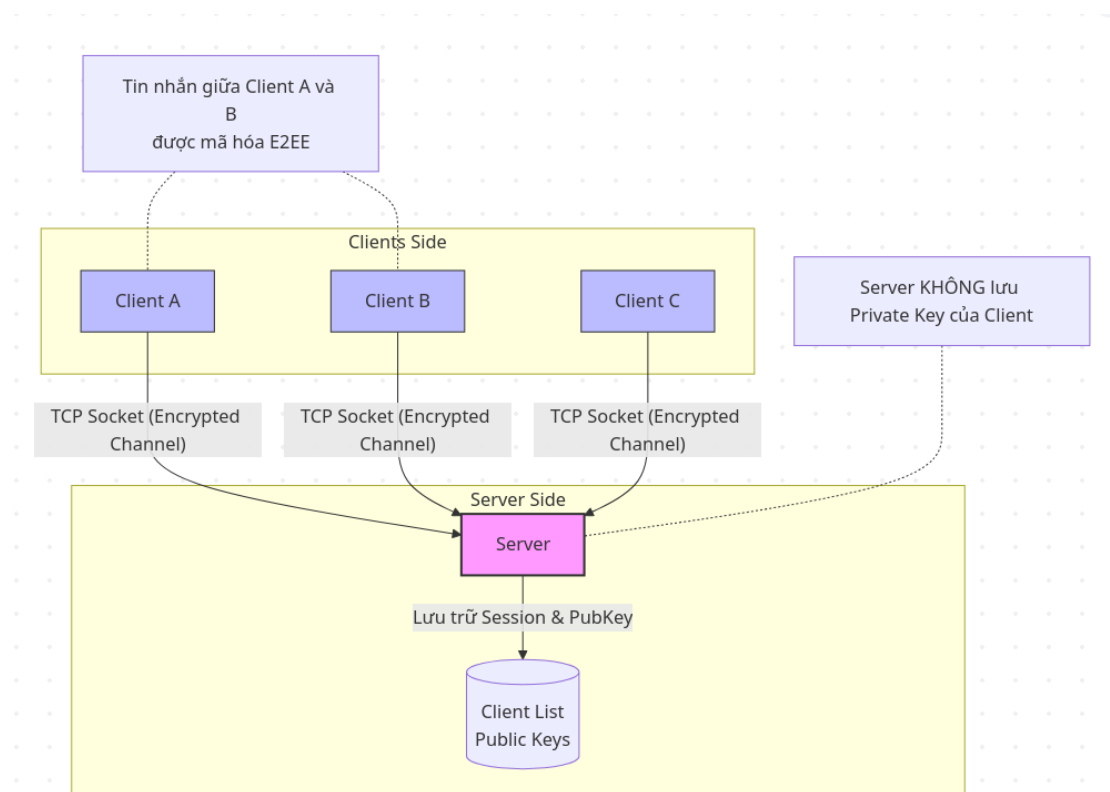
STT	Thuộc tính bảo mật	Mô tả yêu cầu	Cơ chế / Thuật toán áp dụng
2	Tính toàn vẹn (Integrity)	Đảm bảo tin nhắn không bị thay đổi trong quá trình truyền; phát hiện mọi hành vi chỉnh sửa dữ liệu.	Hàm băm SHA-256 kết hợp chữ ký số RSA.
3	Tính xác thực (Authentication)	Xác minh danh tính Server trước khi kết nối và xác thực nguồn gốc của tin nhắn giữa các Client.	Chứng chỉ số Server (CA giả lập); xác thực chữ ký số bằng RSA Public Key.
4	Tính chống chối bỏ (Non-repudiation)	Đảm bảo người gửi không thể phủ nhận việc đã gửi tin nhắn.	Chữ ký số RSA được tạo bằng Private Key của người gửi.

Bảng 2.3: Yêu cầu bảo mật của hệ thống

2.2.2 Yêu cầu về Hiệu năng

Hệ thống không sử dụng RSA để mã hóa toàn bộ nội dung tin nhắn (vì RSA chậm). Thay vào đó, sử dụng RSA để trao đổi khóa AES (nhỏ gọn), và dùng AES để mã hóa nội dung chính. Điều này đảm bảo tốc độ xử lý nhanh ngay cả khi tin nhắn dài.

2.3 Sơ đồ Kiến trúc Hệ thống



Hình 2.1: Sơ đồ kiến trúc hệ thống

Phía Máy chủ (Server Side):

- Server (Trung tâm): Đóng vai trò là bộ định tuyến và chuyển tiếp tin nhắn (Message Relay). Server chịu trách nhiệm lắng nghe các kết nối tại địa chỉ IP và Port đã định
- Lưu trữ: lưu trữ trạng thái phiên làm việc (Session), danh sách người dùng đang online và Khóa công khai (Public Key) của họ.
- Nguyên tắc Zero-Knowledge: Server sẽ không lưu trữ và không có quyền truy cập vào Khóa bí mật (Private Key) của bất kỳ Client nào. Điều này đảm bảo Server không thể giải mã nội dung tin nhắn ngay cả khi bị tấn công hoặc người quản trị cố tình muốn xem.

Phía Máy khách (Clients Side):

- Bao gồm các Client (A, B, C) đại diện cho người dùng cuối.
- Mỗi Client tự quản lý cặp khóa (Public/Private Key) của riêng mình. Private Key được sinh ra và lưu trữ cục bộ tại máy Client, không bao giờ được gửi đi qua mạng.

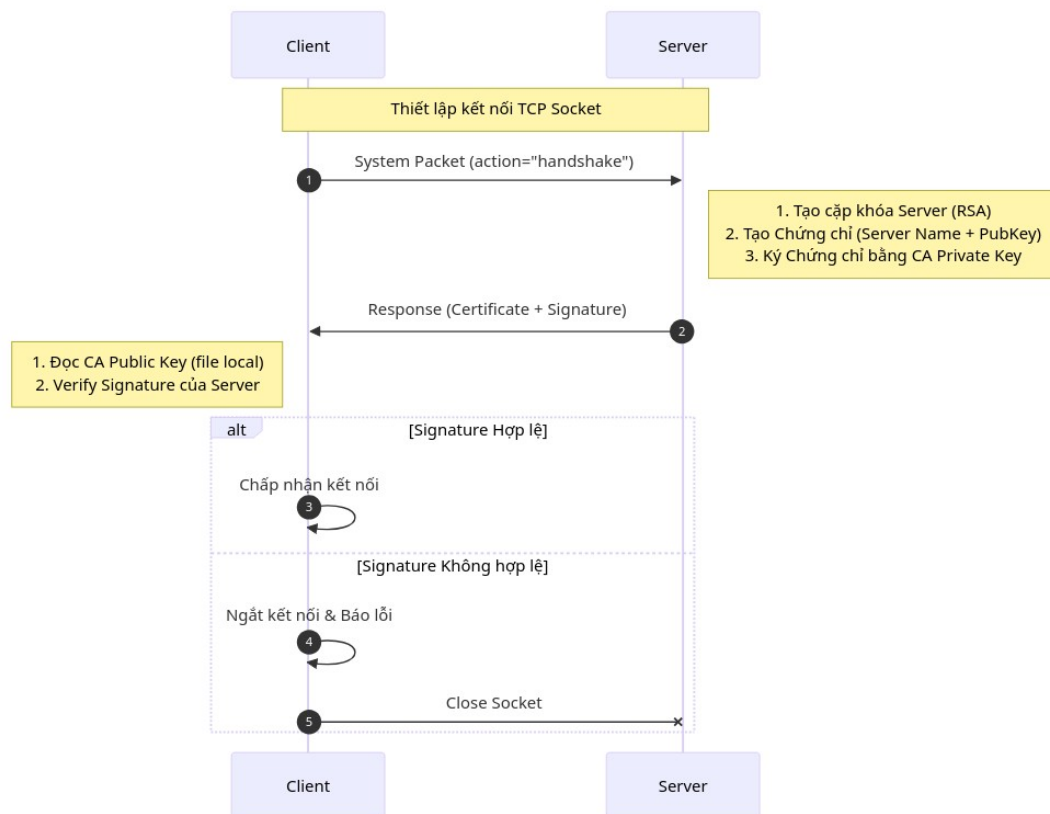
Cơ chế kết nối và Bảo mật:

TCP Socket (Encrypted Channel): Các kết nối từ Client đến Server là các kết nối TCP ổn định. Ngay sau quá trình bắt tay (Handshake), một khóa phiên AES (session_key) được thiết lập để mã hóa toàn bộ đường truyền này, bảo vệ các lệnh hệ thống (như đăng ký, lấy danh sách user).

Mã hóa E2EE: Mặc dù về mặt vật lý, tin nhắn đi từ Client A -> Server -> Client B, nhưng về mặt logic bảo mật, tin nhắn được mã hóa E2EE. Nghĩa là gói tin được khóa tại A và chỉ có thể mở tại B. Đối với Server, gói tin này chỉ là chuỗi dữ liệu vô nghĩa.

2.4 Kịch bản giao tiếp (Protocol Flow)

2.4.1 Quy trình Bắt tay (Handshake) & Xác thực Server



Hình 2.2: Quy trình Bắt tay (Handshake) & Xác thực Server

Client: Gửi gói tin system với action handshake.

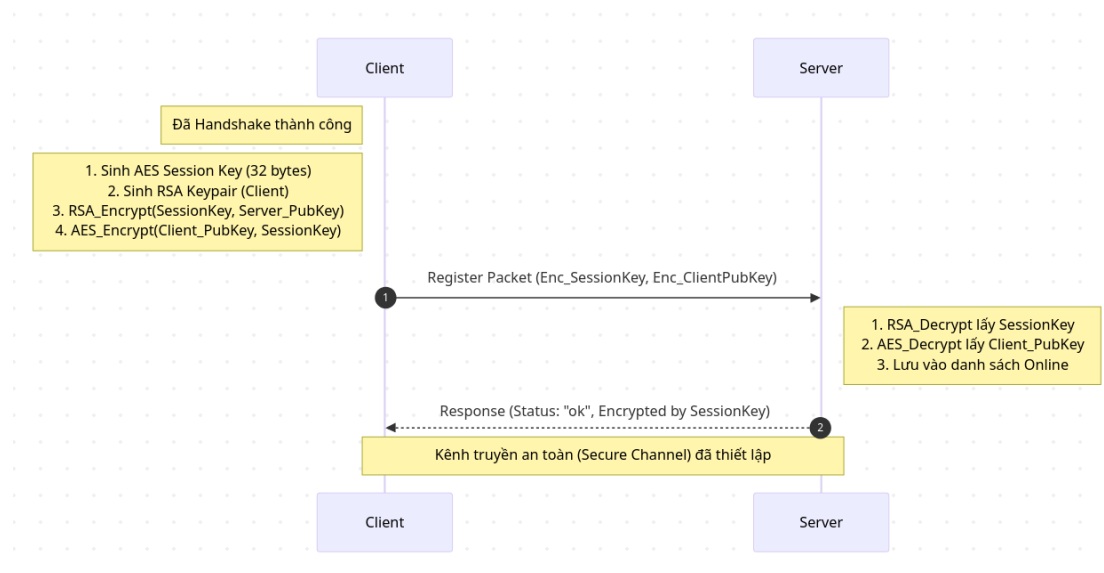
Server: Tự tạo cặp khóa RSA cho CA (giả lập) và cho Server.

- Tạo chứng chỉ (certification) chứa Server Public Key.
- Dùng CA Private Key để ký (Sign) vào chứng chỉ đó.
- Gửi Chứng chỉ + Chữ ký về cho Client.

Client:

- Dùng certification (đã có sẵn) để xác thực (verify signature) chữ ký của Server.
- Nếu đúng: Chấp nhận kết nối. Nếu sai: Ngắt kết nối.

2.4.2 Quy trình Đăng ký & Thiết lập Session Key



Hình 2.3: Quy trình Đăng ký & Thiết lập Session Key

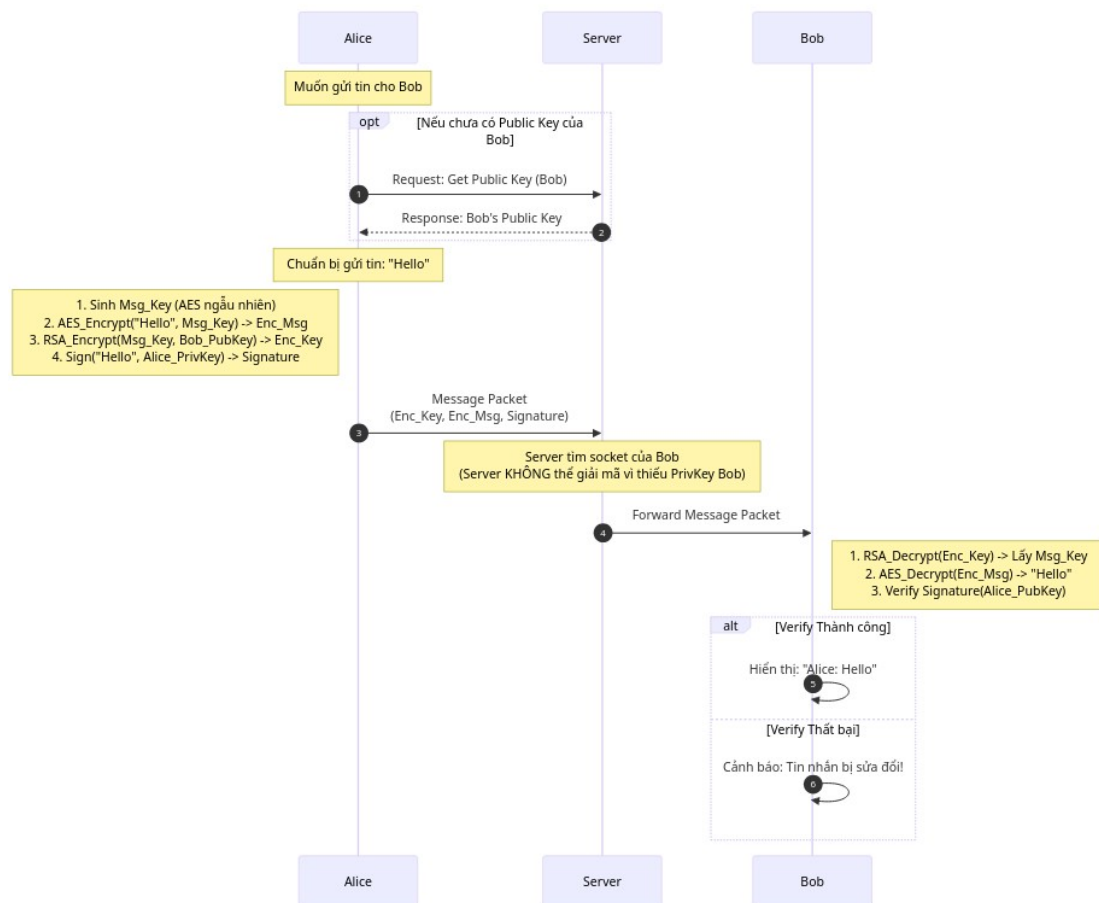
Client:

- Sinh một khóa AES ngẫu nhiên 32 bytes (session_key).
- Mã hóa session_key này bằng Server Public Key (lấy từ bước Handshake).
- Sinh cặp khóa RSA cho bản thân Client.
- Mã hóa Client Public Key bằng session_key.
- Gửi gói tin register chứa: Username + Encrypted Session Key + Encrypted Public Key.

Server:

- Dùng Server Private Key giải mã để lấy session_key.
- Dùng session_key giải mã để lấy Client Public Key.
- Lưu session: {username: session_key, socket, public_key}.

2.4.3 Quy trình Trao đổi tin nhắn (E2EE)



Hình 2.4: Quy trình Trao đổi tin nhắn (E2EE)

Client A:

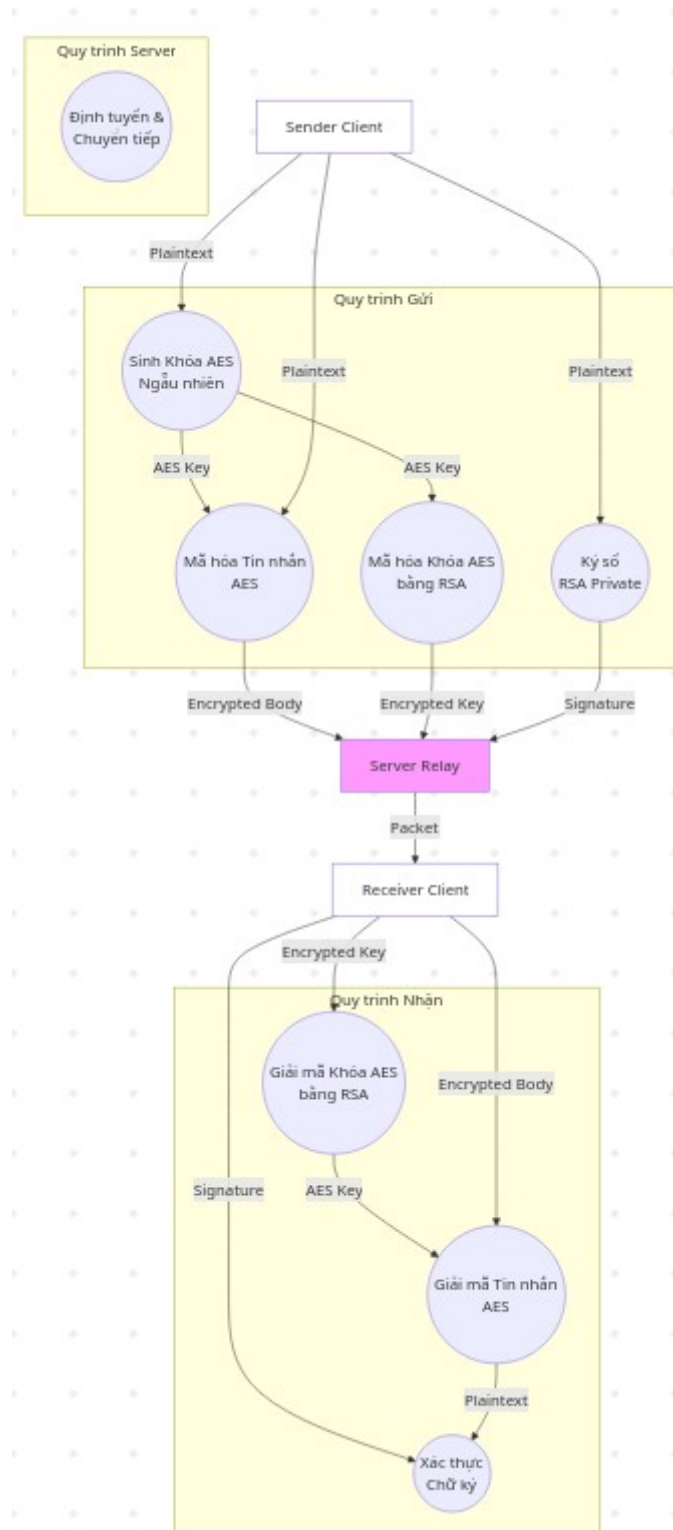
- Yêu cầu Public Key của B từ Server.
- Sinh khóa AES ngẫu nhiên cho tin nhắn (msg_key).
- Mã hóa nội dung tin nhắn bằng msg_key (AES) -> enc_message.
- Mã hóa msg_key bằng Public Key của B (RSA) -> enc_key.
- Tạo chữ ký số (Sign) lên nội dung gốc bằng Private Key của A -> signature.
- Gửi gói tin message: from: A, to: B, enc_key, enc_message, signature.

Server:

- Nhận gói tin.
- Tìm Socket của B.
- Chuyển tiếp (Forward) toàn bộ gói tin sang cho B.

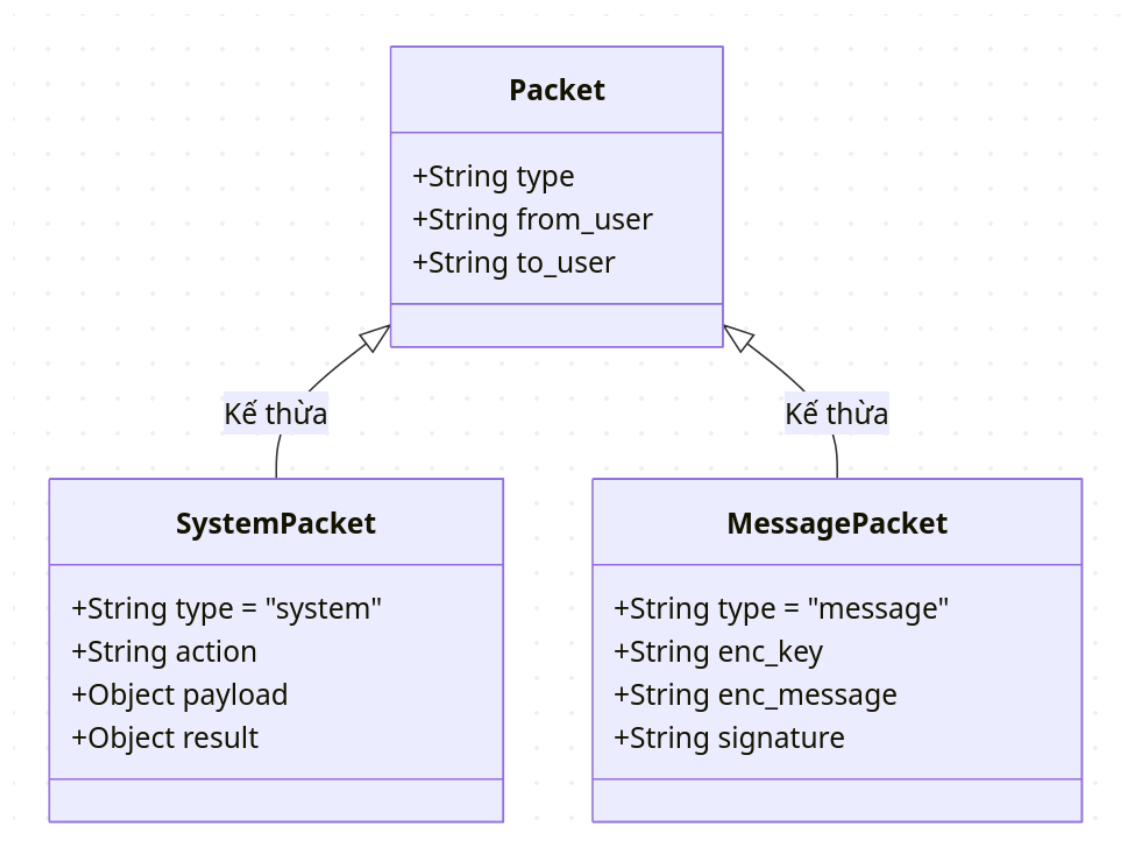
Client B:

- Nhận gói tin.
- Dùng Private Key của B để giải mã enc_key -> lấy được msg_key.
- Dùng msg_key để giải mã enc_message -> lấy được nội dung gốc.
- Lấy Public Key của A (từ cache hoặc hỏi Server) để xác thực signature.



Hình 2.5: Luồng mã hóa và truyền tin E2EE

2.5 Thiết kế cấu trúc gói tin



Hình 2.6: Cấu trúc gói tin

- Gói tin Tin nhắn (Message Packet): Dùng để chat giữa các client.
- Gói tin Hệ thống (System Packet): Dùng cho Handshake, Register, Get Users, Get Public Key.

CHƯƠNG 3: TRIỂN KHAI ĐỀ TÀI

3.1 Quá trình khởi động

```
[2025-12-23 21:47:26] [INFO] Server is starting...  
[2025-12-23 21:47:26] [INFO] Server listening on 127.0.0.1:1234  
[2025-12-23 21:47:26] [SUCCESS] CA key pair and Server key pair generated successfully.
```

Hình 3.1: Quá trình khởi động Server

Trong quá trình khởi động, Server sẽ tự động sinh ra 2 cặp khóa RSA:

- Cặp khóa CA (Certificate Authority): Dùng để ký chứng thực cho Server (giả lập tổ chức cấp phát chứng chỉ).
- Cặp khóa Server: Dùng để Server giải mã các Session Key mà Client gửi lên sau này.

```
Enter your username: Huy  
[2025-12-23 21:54:39] [INFO] Username set to 'Huy'.  
[2025-12-23 21:54:39] [SUCCESS] RSA key pair generated successfully.  
[2025-12-23 21:54:39] [SUCCESS] Connected to server at 127.0.0.1:1234.  
[2025-12-23 21:54:39] [INFO] Sending handshake to server...  
[2025-12-23 21:54:39] [INFO] Authenticating server...
```

Hình 3.2: Quá trình khởi tạo Client và xác thực Server

Khi khởi động Client, hệ thống sẽ yêu cầu nhập username thay cho bước đăng nhập để xác minh danh tính. Sau đó, Client tự động sinh ra cặp khóa RSA (Public/Private Key) cục bộ. Cặp khóa này sẽ được dùng để nhận tin nhắn mã hóa đầu cuối (E2EE) và chữ ký số sau này.

Trong trường hợp username đã được đăng ký trên Server, Server sẽ từ chối kết nối và Client sẽ tự động thoát.

```
[2025-12-23 22:38:49] [ERROR] This username is already taken. Please choose a different one.  
Huy > [2025-12-23 22:38:50] [INFO] Disconnected from the server.
```

Hình 3.3: Server từ chối kết nối nếu trùng username

3.2 Quá trình xác thực Server và thiết lập phiên

Sau khi Client thiết lập kết nối thành công đến Server, Client gửi gói tin yêu cầu bắt tay, phía Server sẽ gửi lại một gói tin phản hồi lại gói tin yêu cầu bắt tay của Client, Server thực hiện đóng gói Chứng chỉ số (Certificate) chứa Public Key của Server và Chữ ký số (Signature) do CA tạo ra. Đây là bước quan trọng để Client có dữ liệu thực hiện việc xác thực.

```
[2025-12-23 21:54:39] [INFO] New client attempting to connect from 127.0.0.1:41064
[2025-12-23 21:54:39] [INFO] Sending handshake response to 'Huy' with server certificate.
```

Hình 3.4: Server tiếp nhận kết nối và phản hồi yêu cầu bắt tay

Sau khi nhận được Chứng chỉ và Chữ ký số từ Server, Client sử dụng khóa công khai của CA (được tin tưởng từ trước) để kiểm tra tính hợp lệ.

```
Huy2 > [2025-12-23 22:11:05] [INFO] Authenticating server...
[2025-12-23 22:11:05] [ERROR] Server authentication failed.
```

Hình 3.5: Client từ chối kết nối khi xác thực Server thất bại

Trong trường hợp chữ ký số gửi từ Server không khớp với chứng chỉ gốc mà Client đang nắm giữ. Ngay sau dòng thông báo lỗi này, Client sẽ tự động ngắt kết nối và thoát chương trình. Hành động này ngăn chặn Client gửi bất kỳ thông tin nhạy cảm nào (như Session Key) vào một môi trường không an toàn.

```
[2025-12-23 21:54:39] [SUCCESS] Server authentication successful.
[2025-12-23 21:54:39] [SUCCESS] Session AES key generated.
[2025-12-23 21:54:39] [INFO] Sending registration request to server...
```

Hình 3.6: Client hoàn tất xác thực và gửi yêu cầu đăng ký phiên làm việc

Bước xác thực Server (Handshake) đóng vai trò then chốt trong việc đảm bảo Tính Xác thực (Authenticity) của hệ thống. Thông qua việc kiểm tra chữ ký số (Digital Signature) dựa trên khóa công khai của CA (Certificate Authority), Client có thể phát hiện và từ chối kết nối ngay lập tức nếu phát hiện dấu hiệu của Server giả mạo. Cơ chế này loại bỏ hoàn toàn nguy cơ bị tấn công bởi kỹ thuật Man-in-the-Middle (MitM), đảm bảo rằng Khóa phiên (Session Key) chỉ được gửi đến đúng Server hợp lệ.

Sau khi khẳng định Server là hợp lệ, ngăn chặn hoàn toàn việc kết nối nhằm vào một Server giả mạo. Client tự động sinh ra một chuỗi ngẫu nhiên 32 bytes (256-bit) làm khóa AES Session Key. Khóa này sẽ được dùng để mã hóa toàn bộ các lệnh hệ thống (như lấy danh sách online, lấy public key ...) sau này, đảm bảo tốc độ nhanh hơn so với RSA.

Client đóng gói một gói tin register bao gồm: Username, Session Key (được mã hóa bằng RSA Server Public Key) và Client Public Key (được mã hóa bằng Session Key).

Việc gửi gói tin này đánh dấu việc hoàn tất quy trình bắt tay (Handshake). Từ thời điểm này, Server sẽ giải mã để lấy Session Key và bắt đầu quản lý phiên làm việc của Client.


```
[2025-12-23 22:22:01] [INFO] Session key for 'Huy' decrypted successfully.  
[2025-12-23 22:22:01] [INFO] Public key for 'Huy' decrypted successfully.  
[2025-12-23 22:22:01] [SUCCESS] User 'Huy' has registered successfully. Session key established.
```

Hình 3.7: Server giải mã khóa và hoàn tất đăng ký người dùng

Server sử dụng RSA Private Key của chính mình để giải mã gói tin chứa Session Key. Điều này đảm bảo tính bảo mật tuyệt đối, vì chỉ có Server mới có Private Key để đọc được khóa phiên AES mà Client vừa sinh ra.

Sau khi có Session Key, Server dùng nó để giải mã tiếp PublicKey để lấy ra RSA Public Key của Client. Server sẽ lưu trữ Public Key này để phục vụ cho tính năng gửi tin nhắn E2EE: Khi Client này muốn nhắn tin cho Client khác đang kết nối, Server sẽ cung cấp Public Key này cho Client đó.

Server lưu thông tin của Client (bao gồm Socket, Session Key, Public Key).

3.3 Giao tiếp hệ thống

3.3.1 Kiểm tra danh sách Client online

```
Huy > online  
  
--- Online Users ---  
1. Huy  
2. Huy2  
-----  
[2025-12-23 22:33:01] [INFO] 2 users currently online.
```

Hình 3.8: Truy vấn danh sách người dùng đang hoạt động

Tại giao diện, người dùng nhập lệnh “online”. Hệ thống sẽ gửi một gói tin yêu cầu đến Server với hành động là Get_Online_Users. Server truy vấn danh sách các kết nối socket đang mở đã được lưu trữ và trả về kết quả. Và gói tin trả về từ Server chứa danh sách này đã được mã hóa bằng AES Session Key của Client đã yêu cầu.

3.3.2 Quá trình gửi tin nhắn

```
Huy > @Huy2 Hello Huy2  
[2025-12-23 22:46:10] [INFO] Public key for 'Huy2' not found. Requesting it from the server...  
[2025-12-23 22:46:11] [INFO] Received public key for 'Huy2'.  
Huy > [2025-12-23 22:46:11] [SUCCESS] Message sent to 'Huy2'.
```

Hình 3.9: Gửi tin nhắn lần đầu và cơ chế tự động tìm kiếm khóa công khai

Ví dụ ta có client “Huy” muốn gửi đến client “Huy2”, nhập lệnh: “@Huy2 Hello Huy2” để gửi tin nhắn. Hệ thống kiểm tra trong bộ nhớ đệm (Cache) xem đã có RSA Public Key của client “Huy2” hay chưa. Do đây là lần đầu tiên hai

người nhắn tin, Client phát hiện chưa có khóa của đối phương. Thay vì báo lỗi, hệ thống tự động tạm dừng việc gửi tin và gửi một yêu cầu truy vấn Get_PublicKey lên Server.

Sau khi nhận được khóa công khai từ Server, Client lưu nó vào Cache (để lần sau không phải hỏi lại). Ngay lập tức, hệ thống lấy khóa vừa nhận được để mã hóa AES Key, sau đó mã hóa tin nhắn bằng AES Key và gửi đi.

3.3.3 Quá trình nhận tin nhắn

```
Huy2 > [2025-12-23 22:46:11] [INFO] Message decrypted from 'Huy'.  
[2025-12-23 22:46:11] [INFO] Waiting for 'Huy' public key to verify the message...  
[2025-12-23 22:46:11] [INFO] Received public key for 'Huy'.  
[2025-12-23 22:46:11] [INFO] Verifying signature from 'Huy'...  
[2025-12-23 22:46:11] [SUCCESS] Signature verified successfully from 'Huy'.  
[Huy] -> you: Hello Huy2
```

Hình 3.10: Quy trình nhận tin Giải mã và Xác thực chữ ký số

Ngay khi nhận gói tin, client "Huy2" đã sử dụng Private Key của mình để giải mã khóa AES, sau đó dùng khóa AES để giải mã nội dung tin nhắn. Lưu ý, tại bước này, tin nhắn đã đọc được ("Hello Huy2") nhưng chưa hiển thị ra màn hình vì chưa qua bước kiểm tra chữ ký số.

Để đảm bảo tin nhắn thực sự đến từ client "Huy" và không bị sửa đổi, hệ thống cần kiểm tra chữ ký số (Signature). Do chưa có Public Key của người gửi (Huy) trong bộ nhớ đệm, Client tự động gửi yêu cầu lên Server để lấy về. Sau khi có Public Key của client "Huy", hệ thống thực hiện thuật toán xác thực (Verify Signature).

Chỉ khi xác minh thành công chữ ký thành công, nội dung "Hello Huy2" mới được xuất hiện trên màn hình. Khẳng định:

- Tin nhắn toàn vẹn (Integrity): Không bị thay đổi dù chỉ 1 bit trên đường truyền.
- Chống chối bỏ (Non-repudiation): Chính xác là client "Huy" gửi chứ không phải ai khác.

Trường hợp tin nhắn bị thay đổi, hệ thống tính toán mã băm (Hash) của nội dung vừa giải mã và so sánh nó với mã băm được giải mã từ chữ ký số của client "Huy".

```
Huy2 > [2025-12-23 23:00:01] [INFO] Message decrypted from 'Huy'.  
[2025-12-23 23:00:01] [INFO] Waiting for 'Huy' public key to verify the message...  
[2025-12-23 23:00:01] [INFO] Received public key for 'Huy'.  
[2025-12-23 23:00:01] [INFO] Verifying signature from 'Huy'...  
[2025-12-23 23:00:01] [ERROR] Signature verification failed for 'Huy'.
```

Hình 3.11: Hệ thống phát hiện và cảnh báo khi tin nhắn bị thay đổi

Kết quả: Hai mã băm không khớp nhau. Điều này chứng tỏ Tính toàn vẹn (Integrity) của tin nhắn đã bị phá vỡ.

3.3.4 Quá trình Server chuyển tiếp tin nhắn mã hóa

```
[2025-12-23 22:46:11] [INFO] User 'Huy' requested the public key of 'Huy2'.
[2025-12-23 22:46:11] [SUCCESS] Sent public key of 'Huy2' to 'Huy'.
[2025-12-23 22:46:11] [SUCCESS] Message relayed from 'Huy' to 'Huy2': Yvhi+gGrlJfMK9CtbSRNTui9h67g00tto19TPIXBSqs=).
[2025-12-23 22:46:11] [INFO] User 'Huy2' requested the public key of 'Huy'.
[2025-12-23 22:46:11] [SUCCESS] Sent public key of 'Huy' to 'Huy2'.
```

Hình 3.12: Server phân phối khóa và Chuyển tiếp tin nhắn mã hóa

Server đóng vai trò là kho chứa khóa công khai. Khi client “Huy” muốn gửi tin, Server đã cung cấp Public Key của client "Huy2" để client “Huy” thực hiện mã hóa.

Nội dung tin nhắn là chuỗi mã hóa (Ciphertext) hoàn toàn vô nghĩa đối với Server. Server chỉ làm nhiệm vụ chuyển gói tin này từ socket của client “Huy” sang socket của client “Huy2” mà không thể giải mã để xem nội dung "Hello Huy2" bên trong.

Điều này khẳng định tính bí mật (Confidentiality) được đảm bảo tuyệt đối. Chỉ có thiết bị của người nhận, nơi lưu trữ Khóa bí mật (Private Key) tương ứng, mới có khả năng giải mã để xem nội dung gốc. Ngay cả quản trị viên hệ thống (Server Admin) cũng không thể can thiệp hay xem trộm nội dung tin nhắn, đảm bảo quyền riêng tư và an toàn thông tin cho người sử dụng.

Sau khi nhận được tin nhắn, client "Huy2" cần xác thực chữ ký số nhưng chưa có khóa của người gửi. Server tiếp tục cung cấp Public Key của client "Huy" cho client "Huy2" để hoàn tất quá trình kiểm tra tính toàn vẹn (Verify Signature) tại phía nhận.

3.4 Chi tiết cấu trúc gói tin

3.4.1 Giai đoạn Bắt tay

```
{
  "type": "system",
  "action": "handshake",
  "from": "Huy",
  "to": "server",
  "payload": { "message": "hello_server" }
}
```

Hình 3.13: Yêu cầu kết nối từ Client

```
{
  "type": "system",
  "action": "handshake",
  "from": "server",
  "to": "Huy",
  "result": {
    "status": "ok",
    "certification": {
      "server_name": "RealServer",
      "public_key": "-----BEGIN PUBLIC KEY-----\nMIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEApd..."
    },
    "signature": "AlVclWWIZVTaV9lr0Ab1H/hV4GvG2lqVfD5pLl1/3kRi5H3y2TSP8MHpsNGHV10v..."
  }
}
```

Hình 3.14: Phản hồi từ Server (Chứa Chứng chỉ & Chữ ký)

3.4.2 Giai đoạn Đăng ký & Trao đổi khóa

```
{
  "type": "system",
  "action": "register",
  "from": "Huy",
  "to": "server",
  "payload": {
    "username": "Huy",
    "session_key": "icKWreP1DtA3atRNTEilQn9z5gdTqhYq6JZA266n8X4Q8FyaNOEiNQZKgu8zZH6...",
    "public_key": "ow0zcczVutQUIUzYqSEEGYqYFZ9K0+tc07FdqaWA0i+IpV32vrikXhwjGXv50C8..."
  }
}
```

Hình 3.15: Client gửi Khóa phiên

session_key: Chứa khóa AES-256 đã được mã hóa bằng RSA Server Public Key.

public_key: Khóa công khai của Client, được mã hóa bằng Session Key vừa sinh ra.

3.4.3 Giai đoạn Giao tiếp Hệ thống

Toàn bộ các lệnh hệ thống đều được bảo vệ bởi khóa AES Session Key.

```
// Request (Client -> Server)
{ "type": "system", "action": "get_online_users", ... "payload": null }

// Response (Server -> Client)
{
  "type": "system", "action": "get_online_users", ...,
  "result": "c4U+EUpGNA6Nl0nxhDAaZsoEsd8PpF8lSyMbIsDEpsq0JGad/LOtbI1UA/mBus1n..."
}
```

Hình 3.16: Truy vấn danh sách Online

Trường result chứa danh sách user (giả sử có "Huy", "Huy2") nhưng đã bị mã hóa thành chuỗi vô nghĩa (c4U+...). Điều này giúp ẩn danh tính người dùng trong mạng.

```
// Request: Client hỏi xin key của một user khác
{ ..., "payload": "Nv3tcnEajxCu+FvMVI4q+9ZLxLd3/q9zYe0qU+Yi5B6gIGCsxgxcM4CwmQrKEPn7" }

// Response: Server trả về Public Key
{ ..., "result": "jK2a2XMtXppuq79NYX2JEobN/xEmh18Y9hVB5mF0EzgWwAMdA4HTwt8us7MRHXez..." }
```

Hình 3.17: Truy vấn Public Key của người khác

3.4.4 Giai đoạn Trao đổi tin nhắn

```
{
  "type": "message",
  "from": "Huy",
  "to": "Huy2",
  "enc_key": "Kv4v42IIH+6gRPe0fq7nsWyip2LQyE6Ugr9ShRnvxPclD0clUozgQsdA0QLeww9U...",
  "enc_message": "owQUGuknCnqIEyGHRhSNjcf+EaeAThfLZOV+vSgajJ4=",
  "signature": "Bj37uvQYvH8B7oCpSEqpgqTWM/QyKKNdqvyKGtS/Woa3NeDr9YMK7VLY7VwqUMto..."
}
```

Hình 3.18: Tin nhắn từ client này gửi tới client khác

Tính Bí mật: Nội dung tin nhắn nằm trong enc_message (owQUG...) hoàn toàn không thể đọc được. Chỉ người có Private Key tương ứng mới giải mã được enc_key để đọc nội dung này.

Tính Toàn vẹn: Trường signature đi kèm đảm bảo tin nhắn không bị sửa đổi.

Vai trò Server: Server chỉ đọc được from và to để định tuyến, không can thiệp vào nội dung.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Sau quá trình nghiên cứu lý thuyết và triển khai ứng dụng, đồ án "Thiết kế và triển khai mã hóa tin nhắn RSA/AES trong mô hình Client-Server" đã hoàn thành các mục tiêu đề ra ban đầu. Ứng dụng không chỉ dừng lại ở việc trao đổi tin nhắn văn bản thông thường mà còn tích hợp thành công các cơ chế bảo mật cốt lõi của an toàn thông tin.

- Xây dựng thành công mô hình Mã hóa lai (Hybrid Encryption): Kết hợp hiệu quả giữa tốc độ của mã hóa đối xứng (AES-256) để mã hóa dữ liệu và tính bảo mật của mã hóa bất đối xứng (RSA-2048) để trao đổi khóa. Điều này giải quyết được bài toán về hiệu năng khi xử lý dữ liệu lớn.
- Thực hiện cơ chế Mã hóa đầu cuối (End-to-End Encryption - E2EE): Đảm bảo tính Bí mật (Confidentiality) tuyệt đối. Server chỉ đóng vai trò trung chuyển (Relay) và lưu trữ khóa công khai, hoàn toàn không có khả năng giải mã nội dung tin nhắn của người dùng.
- Đảm bảo tính Xác thực (Authenticity) và Toàn vẹn (Integrity):
 - + Xác thực Server thông qua cơ chế giả lập CA và Chữ ký số, ngăn chặn tấn công Man-in-the-Middle và Server giả mạo.
 - + Xác thực tin nhắn giữa các người dùng thông qua chữ ký số RSA, giúp phát hiện ngay lập tức nếu gói tin bị sửa đổi trên đường truyền.

Để ứng dụng có thể triển khai trong thực tế và nâng cao trải nghiệm người dùng Trước hết, giao diện dòng lệnh hiện tại có thể được thay thế bằng giao diện đồ họa (GUI) sử dụng các thư viện như Tkinter hoặc PyQt5. Việc này giúp hiển thị danh sách người dùng trực quan hơn, hỗ trợ các cửa sổ chat riêng biệt và cung cấp thông báo khi có tin nhắn mới hoặc sự cố bảo mật.

Bên cạnh đó, hệ thống cần bổ sung cơ chế hỗ trợ tin nhắn offline và lưu trữ lịch sử trò chuyện. Hiện tại, siêu dữ liệu (Metadata) như người gửi, người nhận vẫn được gửi dưới dạng văn bản rõ. Trong tương lai, vấn đề này có thể được giải quyết bằng cách triển khai giao thức TLS/SSL cho lớp kết nối Socket. Khi đó, toàn bộ gói tin (bao gồm cả Header và Payload) sẽ được mã hóa ở tầng vận chuyển, ngăn chặn hoàn toàn khả năng nghe lén của bên thứ ba.

Hệ thống có thể được mở rộng để hỗ trợ chat nhóm, truyền tải tập tin an toàn và mã hóa cả metadata trong kênh truyền. Với chat nhóm, cơ chế khóa nhóm (Sender-Key) có thể được áp dụng để đảm bảo mã hóa đầu cuối trong khi vẫn tối ưu hiệu năng. Đối với truyền file, dữ liệu có thể được chia nhỏ, mã hóa bằng AES và gửi qua Socket, kèm theo cơ chế theo dõi tiến trình gửi và nhận.

TÀI LIỆU THAM KHẢO

- [1] William Stallings, “*Cryptography and Network Security: Principles and Practice*”, 1998
- [2] Niels Ferguson, Bruce Schneier, & Tadayoshi Kohno, “*Cryptography Engineering: Design Principles and Practical Applications*”, Wiley, 2010
- [3] Brandon Rhodes, “*Foundations of Python Network Programming (3rd Edition)*”, 2014